

GVars Annotations & Transforms: Enhancement Opportunities

@Parallel Today vs With GVars

The current implementation is deliberately naive:

- Spawns a **raw thread per iteration** (virtual on JDK 21+, platform on 17)
- No thread pool / bounded parallelism
- No back-pressure — 10,000 iterations = 10,000 threads
- No exception collection
- No waiting for completion
- Fire-and-forget

With `groovy-parallel-collections` integrated, `@Parallel` could be upgraded to use `GVarsPool` / `ForkJoinPool` under the hood instead of `ThreadHelper.startThread()`. The transform would rewrite:

```
@Parallel
for (item in items) { process(item) }
```

into something equivalent to:

```
GVarsPool.withPool {
    items.eachParallel { item -> process(item) }
}
```

This gives you bounded parallelism, work-stealing, proper exception propagation, and completion waiting — all for free. The annotation could also gain attributes:

```
@Parallel(pool=4)           // fixed pool size
@Parallel(collect=true)      // collect results (parallel map)
@Parallel(await=true)        // wait for all to complete (default true)
```

Classpath-sensitive transform: The `@Parallel` annotation stays in core Groovy (it's just the annotation + a thin transform), but when `groovy-parallel-collections` is on the classpath, the transform detects it and uses the pool-backed implementation. Without it, you get the current thread-per-iteration fallback. This is similar to how `@Grab` works — the annotation is in core, the engine is in a submodule.

Existing GVars Annotations That Would Carry Over

@ActiveObject / @ActiveMethod

Module: `groovy-actors`

GVars' most compelling AST transformation. It turns a regular class into an actor-backed object:

```
@ActiveObject
class MyService {
    @ActiveMethod
    def process(data) { /* runs on actor thread */ }
}
```

Every `@ActiveMethod` call is asynchronously dispatched to an internal actor and returns a `DataflowVariable` (or with Groovy `async`, an `Awaitable`). The caller can `await` the result or fire-and-forget.

@AsyncFun

Module: `groovy-parallel-core` OR `groovy-dataflow`

Marks a closure field as an asynchronous function — the closure is wrapped so it returns a `Promise` / `Awaitable` instead of blocking:

```
@AsyncFun
Closure<String> download = { url -> fetchUrl(url) }

def result = download('http://example.com') // returns Promise
```

Could be updated to return `Awaitable` with the `async` proposal.

New Annotations/Transforms That GVars Integration Enables

@Dataflow on variables

Module: `groovy-dataflow`

A statement-level annotation that turns a local variable into a `DataflowVariable`:

```
@Dataflow def x
@Dataflow def y
@Dataflow def z

async { x << 10 }
async { y << 20 }
async { z << x.val + y.val } // automatically waits for x and y

assert z.val == 30
```

Or with the `async` proposal integration:

```
@Dataflow def x
async { x << expensiveComputation() }
def result = await x
```

@Agent on fields

Module: `groovy-agents`

Turns a field into a thread-safe agent:

```
class Counter {
    @Agent int count = 0

    void increment() { count { it + 1 } } // mutation submitted as closure
    int getValue() { count.val }
}
```

@Parallel on methods

Module: core Groovy + `groovy-parallel-collections`

Extend beyond loops to methods, running the method body on a pool thread and returning a future/awaitable:

```
@Parallel
def fetchData(url) {
    // runs on pool thread, caller gets Awaitable
    new URL(url).text
}

def result = await fetchData('http://example.com')
```

Summary: Annotation Placement

Annotation	Lives In	Transform Uses	Falls Back To
<code>@Parallel</code> (loop)	core Groovy	<code>groovy-parallel-collections</code> (GVarsPool)	ThreadHelper (current)
<code>@ActiveObject</code>	<code>groovy-actors</code>	Actor infrastructure	n/a
<code>@ActiveMethod</code>	<code>groovy-actors</code>	Actor infrastructure	n/a
<code>@AsyncFun</code>	<code>groovy-parallel-core</code> OR <code>groovy-dataflow</code>	Pool + Promise/Awaitable	n/a
<code>@Dataflow</code>	<code>groovy-dataflow</code>	DataflowVariable	n/a
<code>@Agent</code>	<code>groovy-agents</code>	Agent infrastructure	n/a

Highest-value addition: The `@ActiveObject` transform is probably the highest-value addition from GVars' annotation side — it's a genuinely unique capability that no other JVM language offers as a compile-time annotation.